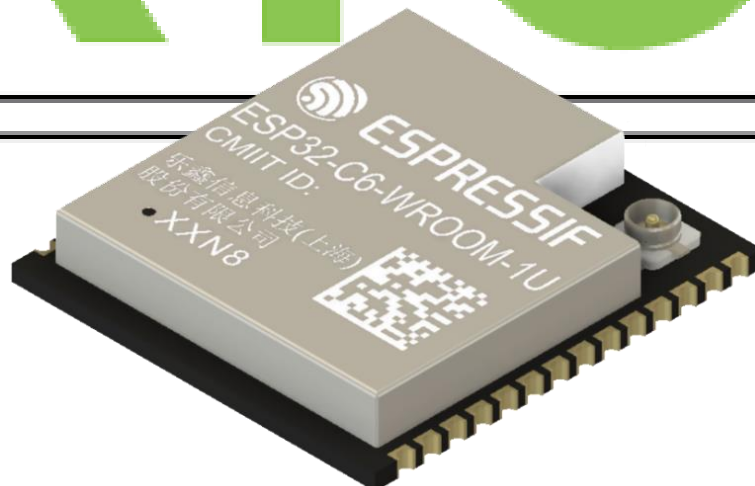


30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 11)
“Counting
Semaphores”

freeRTOS



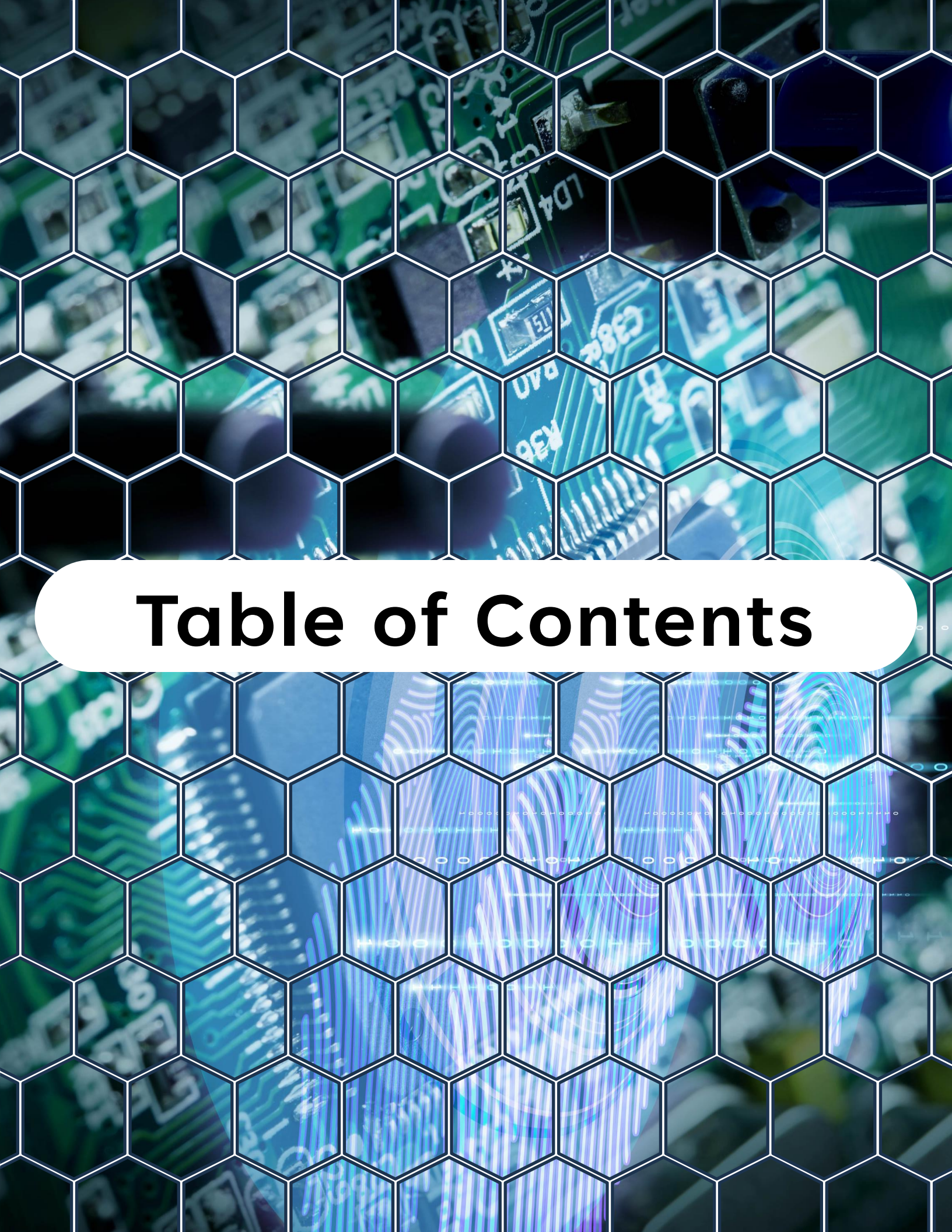
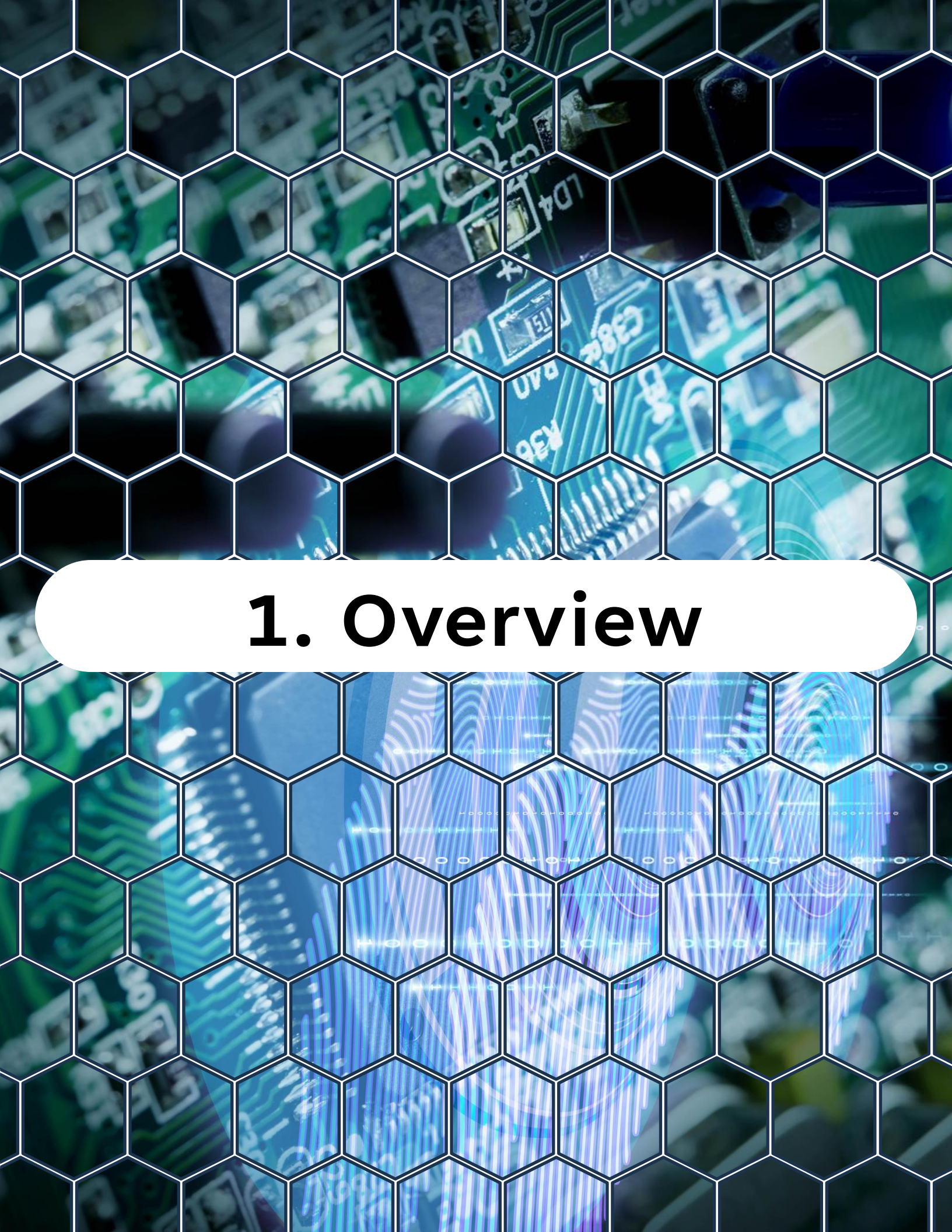


Table of Contents

Table of Contents

1. Overview
2. What Is a Counting Semaphore?
3. Counting Semaphore Functions
4. Binary vs Counting Semaphores
5. Example – Resource Pool with Counting Semaphore
6. Real-World Use Cases
7. Best Practices
8. Summary
9. Challenge for Today



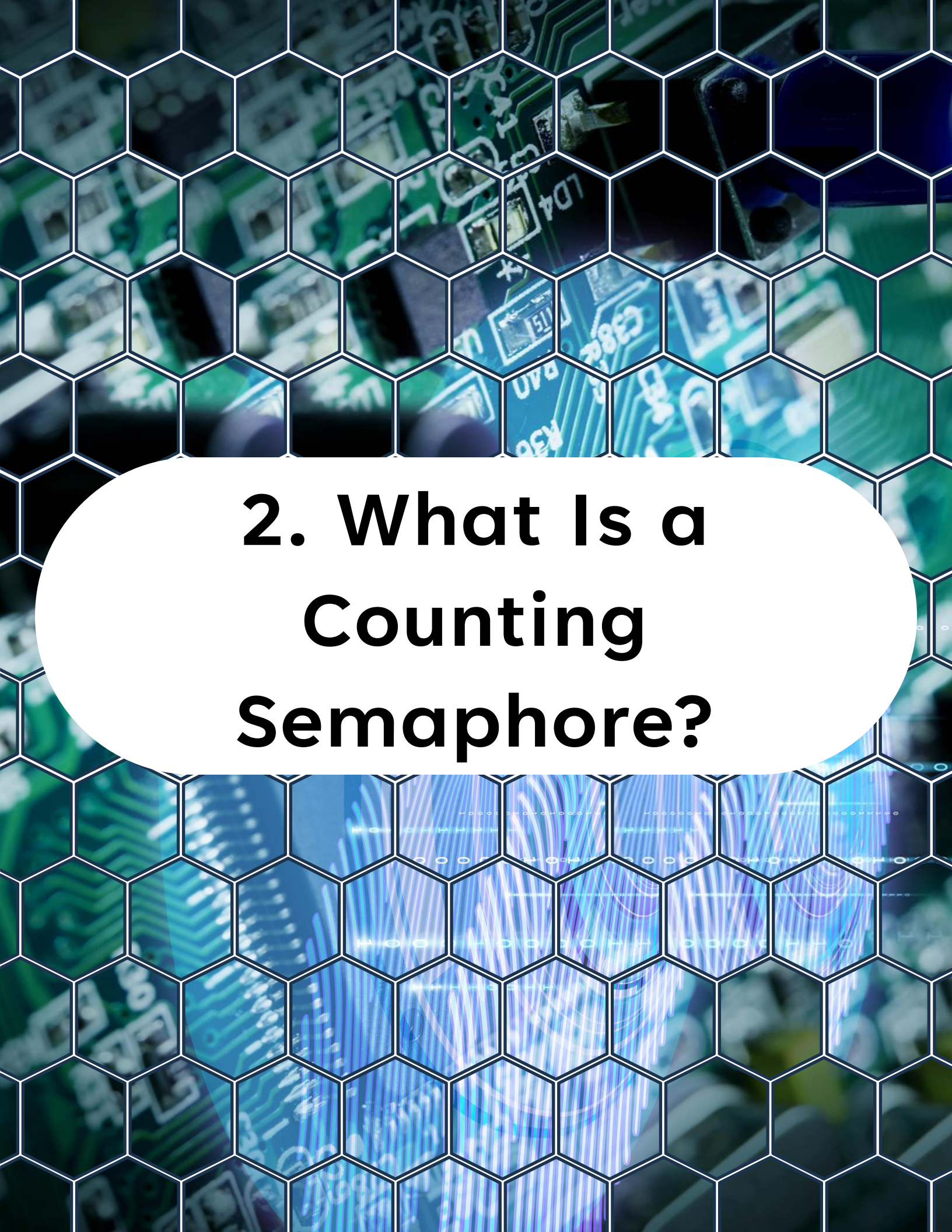
1. Overview

1. Overview

On **Day 11**, you'll learn:

- What **counting semaphores** are in FreeRTOS
- How they extend the idea of binary semaphores
- Common use cases for counting semaphores
- How to create, take, and give them
- A practical ESP32 example: **managing multiple resources**
- Best practices for avoiding misuse

By the end of this lesson, you'll understand how to use counting semaphores to control access to multiple resources or track events.



2. What Is a Counting Semaphore?

2. What Is a Counting Semaphore?

A counting semaphore is like a binary semaphore, but instead of just two states (0 or 1), it maintains a counter.

- **Count > 0** → resource available / events pending
- **Count = 0** → no resources available / no events pending

Every time a task or ISR gives the semaphore, the counter increments.

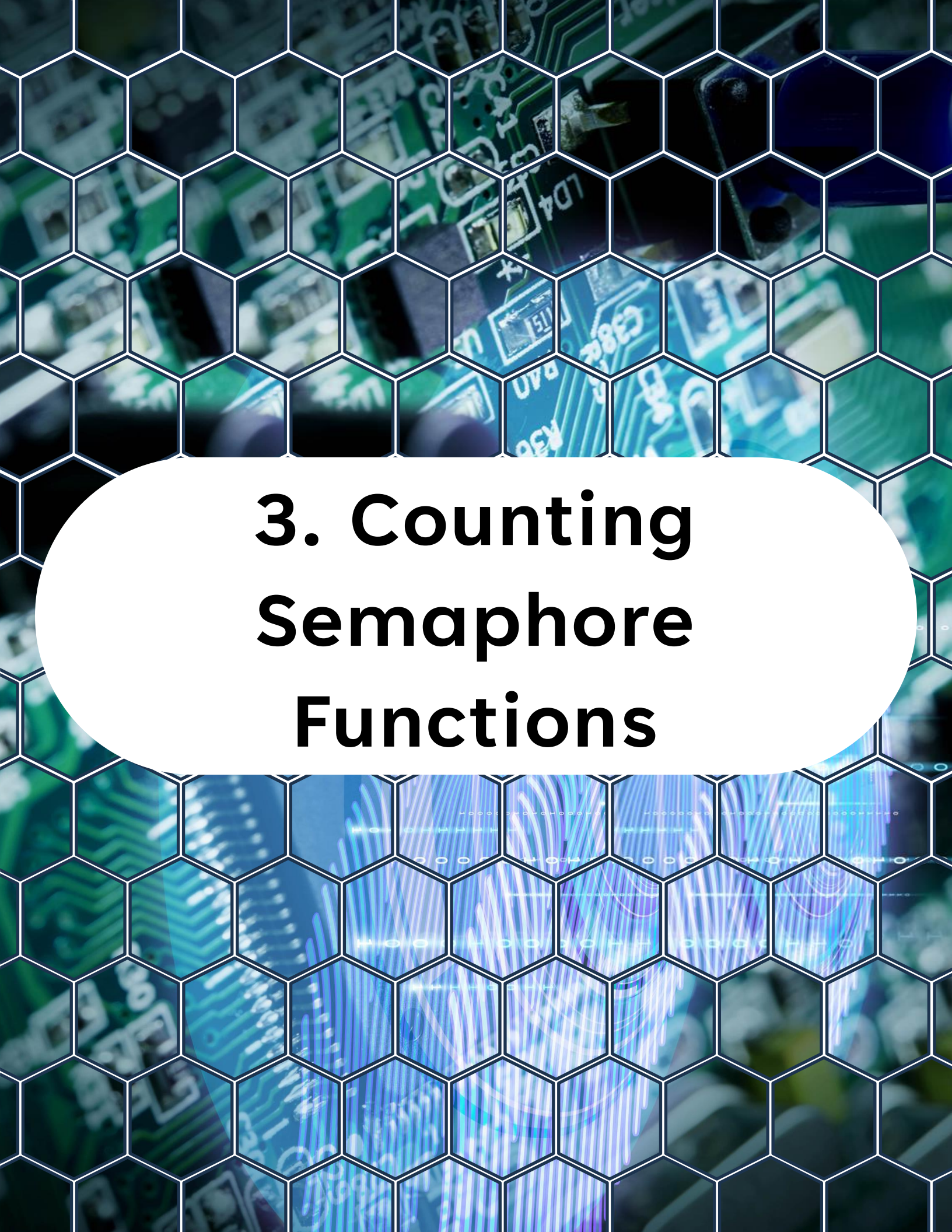
Every time a task takes the semaphore, the counter decrements.

2. What Is a Counting Semaphore?



Key uses:

- Managing access to a pool of identical resources (e.g., buffers, connections).
- Counting the number of times an event occurred, so tasks can catch up later.



3. Counting Semaphore Functions

3. Counting Semaphore Functions

Creating a Counting Semaphore



```
1 SemaphoreHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount,  
2                                             UBaseType_t uxInitialCount);
```

uxMaxCount: maximum value of the

counteruxInitialCount: starting value

Giving (incrementing)

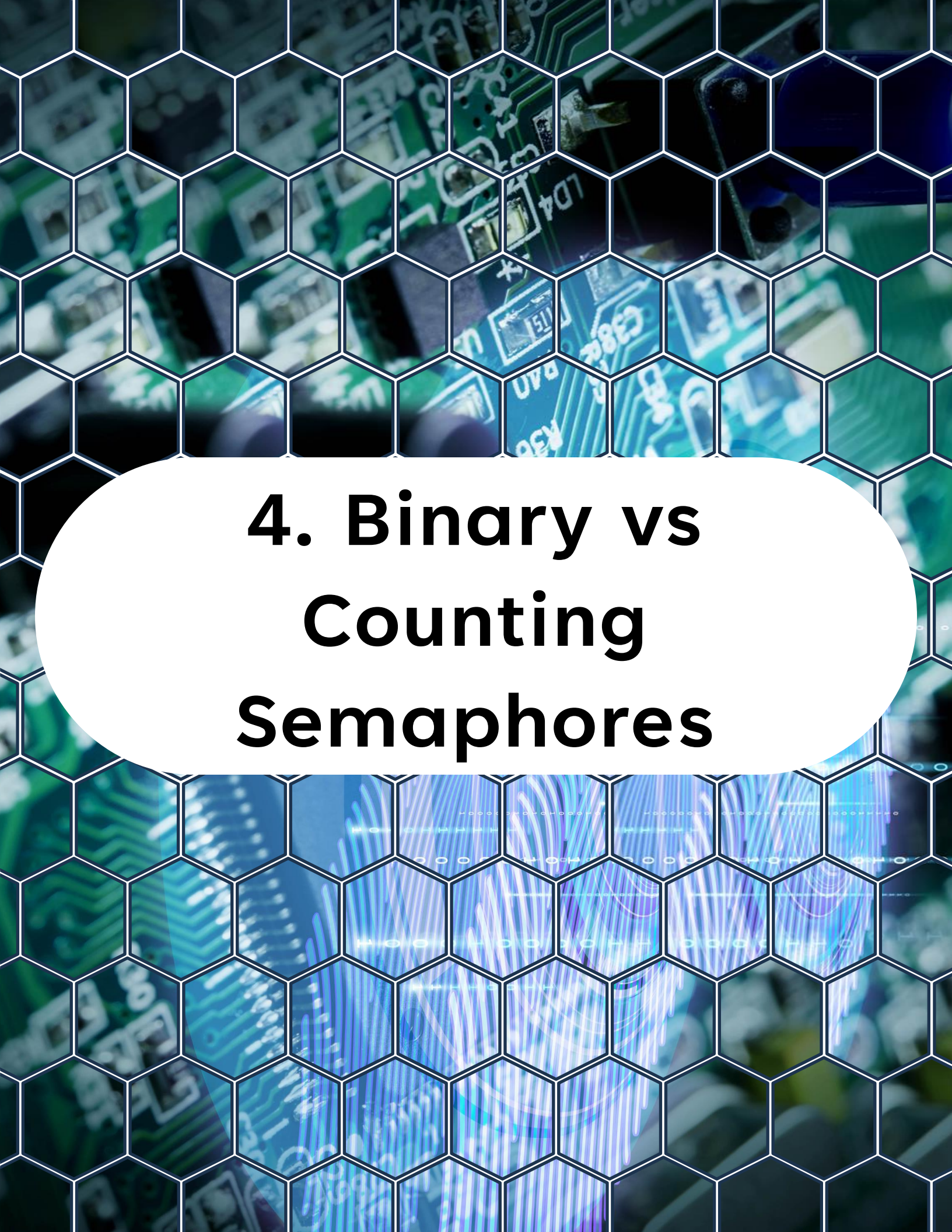


```
1 xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

Taking (decrementing)



```
1 xSemaphoreTake(SemaphoreHandle_t xSemaphore,  
2               TickType_t xTicksToWait);
```

4. Binary vs Counting Semaphores

4. Binary vs Counting Semaphores

Feature	Binary Semaphore	Counting Semaphore
Values	0 or 1	0 up to max count
Event storage	Only 1 event	Multiple events can be stored
Resource management	Single resource	Multiple resources
Typical use	ISR-to-task signaling	Buffer pool, multi-instance resource



5. Example – Resource Pool with Counting Semaphore

5. Example – Resource Pool with Counting Semaphore

Imagine you have **3 UART buffers** available. Only 3 tasks can use them at the same time. A counting semaphore ensures no more than 3 tasks enter the critical section.

Code Example

```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "freertos/semphr.h"
5
6  #define TAG "DAY11"
7  #define MAX_RESOURCES 3
8
9  SemaphoreHandle_t resource_sem;
10
11 void worker_task(void *pvParameters) {
12     int id = (int) pvParameters;
13
14     while (1) {
15         // Wait until a resource is available
16         if (xSemaphoreTake(resource_sem, portMAX_DELAY)) {
17             printf("Task %d acquired resource\n", id);
18
19             // Simulate work with resource
20             vTaskDelay(pdMS_TO_TICKS(2000));
21
22             printf("Task %d releasing resource\n", id);
```

5. Example – Resource Pool with Counting Semaphore

```
14  while (1) {
15      // Wait until a resource is available
16      if (xSemaphoreTake(resource_sem, portMAX_DELAY)) {
17          printf("Task %d acquired resource\n", id);
18
19          // Simulate work with resource
20          vTaskDelay(pdMS_TO_TICKS(2000));
21
22          printf("Task %d releasing resource\n", id);
23
24          // Release resource
25          xSemaphoreGive(resource_sem);
26      }
27      vTaskDelay(pdMS_TO_TICKS(1000));
28  }
29 }
30
31 void app_main() {
32     // Create a counting semaphore with max 3 resources,
33     // initially all available
34     resource_sem = xSemaphoreCreateCounting(MAX_RESOURCES,
35         MAX_RESOURCES);
36
37     if (resource_sem == NULL) {
38         printf("Failed to create counting semaphore\n");
39         return;
40     }
41
42     // Create 5 tasks competing for 3 resources
43     for (int i = 1; i <= 5; i++) {
44         char task_name[16];
45         sprintf(task_name, "Worker%d", i);
46         xTaskCreate(worker_task, task_name, 2048, (void *) i, 5, NULL);
47     }
48 }
49
```


5. Example – Resource Pool with Counting Semaphore

Expected Behavior:

- At most 3 tasks will print “acquired resource” simultaneously.
- Other tasks will wait until one of the resources is released.
- The output will look like:

Task 1 acquired resource

Task 2 acquired resource

Task 3 acquired resource

Task 4 waiting...

Task 5 waiting...

Task 1 releasing resource

Task 4 acquired resource

...



6. Real-World Use Cases

6. Real-World Use Cases

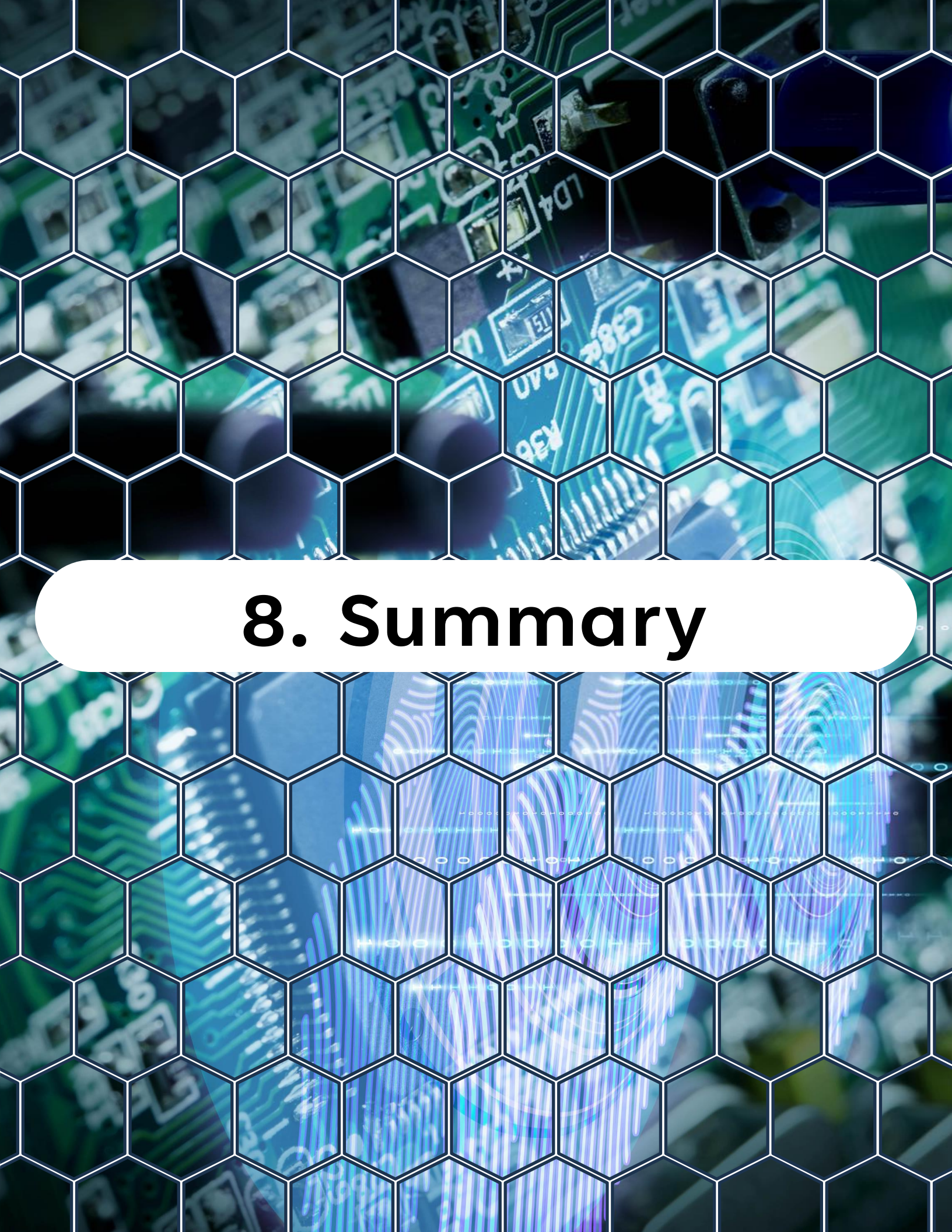
1. **Buffer pools** – e.g., multiple DMA buffers shared between tasks
2. **Peripheral access** – e.g., limit tasks using UART/SPI simultaneously
3. **Event counting** – count the number of interrupts/events before a task wakes up



7. Best Practices

7. Best Practices

- Use **counting semaphores** when **multiple instances of a resource** exist.
- For a **single resource**, prefer **mutex** or **binary semaphore**.
- Initialize with correct **max and initial counts** — misuse can cause deadlocks.
- Avoid making the **max count** too large — it may hide logic bugs.



8. Summary

8. Summary

- ✓ Counting semaphores extend binary semaphores with a **counter**.
- ✓ Perfect for **managing multiple resources** or counting events.
- ✓ Only tasks decrement (take), ISRs or tasks can increment (give).
- ✓ Use carefully to avoid deadlocks and resource leaks.



9. Challenge for Today

9. Challenge for Today

Build a **producer-consumer system**:

1. A producer task simulates an ISR and **gives the semaphore** every 500 ms.
2. Two consumer tasks wait on the semaphore and print when they consume it.
3. Observe how the counting semaphore allows multiple events to queue up if consumers are slow.

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>